



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش ششم

طراحی یک انگوباتور (Incubator)

خلاصه

در این آزمایش واحد کنترل دیجیتال یک سیستم انگوباتور طراحی و پیاده‌سازی شده است. طبق صورت آزمایش، حسگر، Heater و Cooler از مدار قرار دارند؛ بنابراین دما به صورت یک عدد ۸ بیتی وارد مدار می‌شود و عملگرها با سیگنال‌های خروجی نمایش داده می‌شوند که روی برد FPGA به LED و Seven Segment وصل می‌شوند. واحد کنترل از دو ماشین حالت تشکیل شده است که دقیقاً از دو نمودار صورت آزمایش گرفته شده‌اند:

۱. ماشین اصلی که تصمیم می‌گیرد Heater و Cooler روشن باشند یا خاموش (نمودار سمت چپ).
 ۲. ماشین سرعت پنکه که در صورت روشن بودن Cooler، دور Fan را روی ۴، ۶ یا ۸ دور بر ثانیه تنظیم می‌کند (نمودار سمت راست).
- نکته‌ی مهم این است که ماشین دوم فقط وقتی فعال است که ماشین اول در حالت S_2 باشد؛ چون تنظیم دور پنکه‌ای که خاموش است بی‌معنی است.

نمایش دما و نرخ نمونه‌برداری

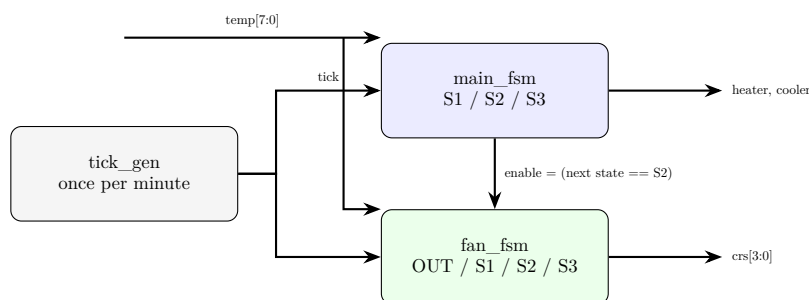
صورت آزمایش می‌گوید حسگر دمای بین -10 تا $+60$ درجه‌ی سانتیگراد را در قالب یک عدد ۸ بیتی تحویل می‌دهد و دما هر یک دقیقه یک بار خوانده می‌شود. برای نمایش دما از کد علامت‌دار مکمل ۲ استفاده شده است، یعنی خود عدد بر حسب درجه:

توضیح	مقدار	کد ۸ بیتی
سردترین مقدار قابل خواندن	-۱۰	8'hF6
صفر درجه	۰	8'h00
گرم‌ترین مقدار قابل خواندن	+۶۰	8'h3C

مزیت این انتخاب آن است که همه‌ی مقایسه‌ها دقیقاً همان‌طور نوشته می‌شوند که در نمودار حالت آمده‌اند؛ یعنی $temp > 35$ به جای مقایسه با یک کد میانی که باید جداگانه محاسبه شود.

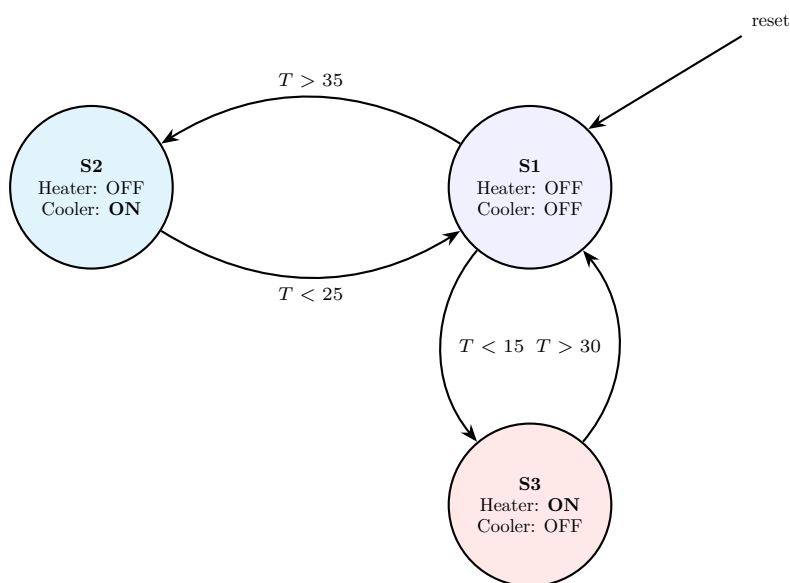
برای نمونه‌برداری یک دقیقه‌ای، ماژول tick_gen یک پالس تک‌کلاکی به نام tick تولید می‌کند و هر دو ماشین حالت فقط با این پالس یک قدم جلو می‌روند. نسبت تقسیم به صورت پارامتر تعریف شده تا در سنتز مقدار واقعی و در شبیه‌سازی یک مقدار کوچک استفاده شود؛ در غیر این صورت تست‌بنچ باید چندین دقیقه زمان واقعی را شبیه‌سازی می‌کرد.

معماری کلی



مسئولیت	ماژول
ماشین حالت اصلی؛ خروجی‌های heater و cooler	main_fsm.v
ماشین حالت سرعت پنکه؛ خروجی crs بر حسب RPS	fan_fsm.v
تولید پالس نمونه‌برداری یک دقیقه‌ای	tick_gen.v
تبدیل عدد سرعت به کد هفت‌قطعه‌ای	seven_seg_decoder.v
اتصال همه‌ی این‌ها به یکدیگر	incubator_controller.v

ماشین حالت اصلی



نمودار فقط برای همین چهار گذار برچسب دارد. برای دمایی که هیچ‌کدام از شرط‌های خروجی حالت فعلی را برآورده نکند، ماشین در همان حالت می‌ماند. این دقیقاً چیزی است که به سیستم خاصیت پسماند (hysteresis) می‌دهد: یک باند مرده‌ی بین ۱۵ تا ۳۵ درجه که جلوی روشن و خاموش شدن پیایی گرم‌کن و خنک‌کن حول یک نقطه‌ی تنظیم را می‌گیرد.

توجه کنید که همه‌ی مقایسه‌ها اکید هستند. مثلاً شرط گذار به S_2 عبارت است از $T > 35$ ، بنابراین در دمای دقیقاً ۳۵ درجه خنک‌کن روشن نمی‌شود. همین نکته در تست‌بنچ برای هر چهار آستانه جداگانه بررسی شده است.

```

1 `include "incubator_pkg.vh"
2
3 // Incubator main controller - LEFT-HAND state diagram.
4 //
5 //           T<25

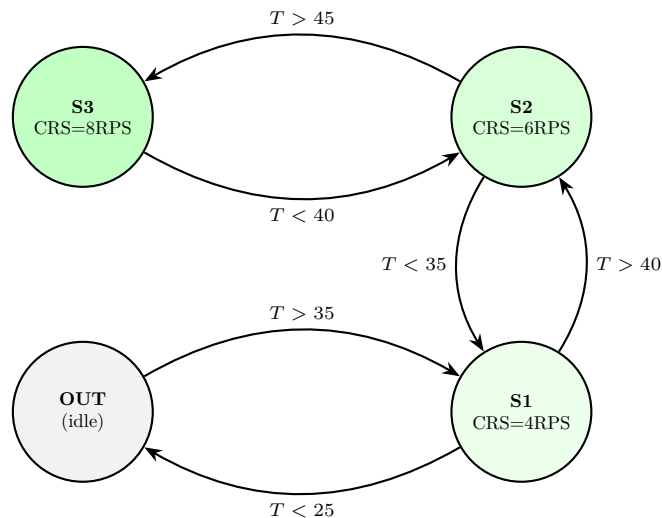
```

```

6 //
7 //
8 //      T>35
9 //      S2          S1      reset
10 //      Heater:      Heater:
11 //      OFF          OFF
12 //      Cooler:      Cooler:
13 //      ON           OFF
14 //
15 //
16 //      T>30          T<15
17 //
18 //      S3
19 //      Heater: ON
20 //      Cooler:
21 //      OFF
22 //
23 //
24 // The diagram labels only the transitions listed above. For any temperature
25 // that does not satisfy the outgoing guard of the current state, the machine
26 // HOLDS its state - that is what gives the controller its hysteresis (the
27 // 15..35 dead band around S1) and stops the heater and cooler chattering on
28 // and off around a single set point.
29 //
30 // `tick` is the once-per-minute sample strobe: the lab specifies that the
31 // temperature is read from the sensor once every minute, so the FSM advances
32 // only on a tick, not on every clock.
33 module main_fsm (
34     input wire          Clk,
35     input wire          RstN,          // async, active-low
36     input wire          tick,          // 1-clock strobe, once per minute
37     input wire signed [7:0] temp,      // degrees C, -10 .. +60
38
39     output wire          heater,        // 1 = heater energised
40     output wire          cooler,        // 1 = cooler energised
41     output wire [1:0]    state,         // current state (debug/waveforms)
42     output wire          cooler_next    // 1 if THIS tick moves us into S2
43 );
44     localparam [1:0] S1 = 2'd0,        // both off (reset state)
45                     S2 = 2'd1,        // cooler on
46                     S3 = 2'd2;        // heater on
47
48     reg [1:0] cur, nxt;
49
50     assign state = cur;
51     // Moore outputs: they depend on the state alone.
52     assign heater = (cur == S3);
53     assign cooler = (cur == S2);
54
55     // The state this FSM will hold *after* the current tick. The fan machine
56     // is gated on this rather than on `cooler`, so that on the very tick that
57     // leaves S2 the fan parks in OUT instead of taking one more step with a
58     // cooler that is already switching off.
59     assign cooler_next = (nxt == S2);
60
61     always @(*) begin
62         nxt = cur;          // default: hold (see note above)
63         case (cur)
64             S1: begin
65                 // Only one of these can be true: 15 < 35, so the guards are
66                 // mutually exclusive and the priority here is cosmetic.
67                 if (temp > `TH_COOL_ON)    nxt = S2;    // T > 35 -> cool
68                 else if (temp < `TH_HEAT_ON)  nxt = S3;    // T < 15 -> heat
69             end
70             S2: if (temp < `TH_COOL_OFF)    nxt = S1;    // T < 25 -> off
71             S3: if (temp > `TH_HEAT_OFF)    nxt = S1;    // T > 30 -> off
72             default:                        nxt = S1;
73         endcase
74     end
75
76     always @(posedge Clk or negedge RstN) begin
77         if (!RstN)      cur <= S1;
78         else if (tick)  cur <= nxt;
79     end
80 endmodule

```

ماشین حالت سرعت پنکه



طبق متن صورت آزمایش، این ماشین تنها زمانی فعال است که ماشین اصلی در حالت S_1 باشد و در غیر این صورت وارد حالت OUT خود می‌شود. بنابراین سیگنال enable که پایین می‌آید، حالت را مجبور به OUT می‌کند، نه اینکه صرفاً آن را منجمد نگه دارد. نتیجه این است که هر بار ورود دوباره به S_1 ، پنکه از پایین‌ترین پله‌ی نردبان سرعت شروع می‌کند، نه از سرعتی که قبلاً داشته است.

```

1  `include "incubator_pkg.vh"
2
3  // Cooler fan-speed controller - RIGHT-HAND state diagram.
4  //
5  //          T<40
6  //
7  //
8  //
9  //          S3          T>45          S2
10 //          CRS=8RPS      CRS=6RPS
11 //
12 //
13 //          T>40          T<35
14 //
15 //          OUT          T>35          S1
16 //          (idle)      CRS=4RPS
17 //
18 //          T<25
19 //
20 // Enable rule (from the lab text)
21 // -----
22 // This machine is active ONLY while the main FSM sits in its S2 state, i.e.
23 // while the cooler is actually running - setting a rotation speed for a
24 // cooler that is switched off is meaningless. The lab sheet states that when
25 // the main FSM is not in S2 this machine "enters its OUT state", so `enable`
26 // going low forces OUT rather than freezing the current state. A later
27 // re-entry into S2 therefore always restarts from OUT.
28 //
29 // As in the main FSM, a temperature that satisfies no outgoing guard leaves
30 // the state unchanged.
31 module fan_fsm (
32     input wire          Clk,
33     input wire          RstN,      // async, active-low
34     input wire          tick,      // once-per-minute sample strobe
35     input wire signed [7:0] temp,
36     input wire          enable,    // 1 only while main FSM is in S2
37
38     output wire [3:0]    crs,      // cooler rotational speed, in RPS
39     output wire [1:0]    state    // exposed for debug/waveforms
40 );
41 localparam [1:0] OUT  = 2'd0,    // fan idle (cooler is off)
42                F_S1 = 2'd1,    // 4 RPS
43                F_S2 = 2'd2,    // 6 RPS
44                F_S3 = 2'd3;    // 8 RPS
45
46 reg [1:0] cur, nxt;
47
48 assign state = cur;
49

```

```

50 // Moore output: the speed is a property of the state alone.
51 // OUT reports 0 RPS because the fan is not turning at all then.
52 assign crs = (cur == F_S1) ? `CRS_4 :
53             (cur == F_S2) ? `CRS_6 :
54             (cur == F_S3) ? `CRS_8 : `CRS_OFF;
55
56 always @(*) begin
57     nxt = cur; // default: hold
58     case (cur)
59     OUT: if (temp > `TH_FAN_IN)    nxt = F_S1; // T > 35
60     F_S1: begin
61         if (temp > `TH_FAN_UP1) nxt = F_S2; // T > 40
62         else if (temp < `TH_FAN_OUT) nxt = OUT; // T < 25
63     end
64     F_S2: begin
65         if (temp > `TH_FAN_UP2)    nxt = F_S3; // T > 45
66         else if (temp < `TH_FAN_DOWN1) nxt = F_S1; // T < 35
67     end
68     F_S3: if (temp < `TH_FAN_DOWN2) nxt = F_S2; // T < 40
69     default:                        nxt = OUT;
70     endcase
71 end
72
73 always @(posedge Clk or negedge RstN) begin
74     if (!RstN) cur <= OUT;
75     else if (!enable) cur <= OUT; // cooler off => park in OUT
76     else if (tick) cur <= nxt;
77 end
78 endmodule

```

نکته‌ی طراحی: هم‌زمانی دو ماشین حالت

مهم‌ترین نکته‌ی ظریف این آزمایش، لحظه‌ی دقیق فعال یا غیرفعال شدن ماشین دوم است.

اگر سیگنال enable را مستقیماً به خروجی cooler (یعنی حالت فعلی ماشین اصلی) وصل کنیم، در همان tick که ماشین اصلی از S_2 خارج می‌شود، مقدار cooler هنوز مقدار قدیمی است. در نتیجه ماشین پنکه یک قدم اضافه برمی‌دارد و برای یک دوره‌ی کامل نمونه‌برداری (یعنی یک دقیقه‌ی واقعی) وضعیتی نمایش داده می‌شود که در آن خنک‌کن خاموش است ولی پنکه هنوز در حال چرخش است. این دقیقاً همان چیزی است که متن آزمایش می‌گوید بی‌معنی است.

راه‌حل این است که ماشین پنکه با حالت بعدی ماشین اصلی گیت شود:

```

// in main_fsm: the state this FSM will hold AFTER the current tick
assign cooler_next = (nxt == S2);

// in incubator_controller: gate the fan on the next state, not the current one
fan_fsm u_fan ( ... .enable(cooler_next), ... );

```

با این تغییر، همان tick که خنک‌کن را خاموش می‌کند، پنکه را هم در OUT پارک می‌کند و هر دو ماشین بر اساس یک نمونه‌ی دما به‌صورت سازگار عمل می‌کنند. در سمت ورود هم رفتار درست است: چون شرط $T > 35$ باعث ورود ماشین اصلی به S_2 می‌شود و هم شرط گذار OUT به S_1 در ماشین پنکه است، خنک‌کن در همان لحظه‌ی روشن شدن سرعت اولیه‌ی ۴ RPS خود را می‌گیرد.

این اشکال در اولین اجرای تست‌بنچ دیده نشد، چون مدل مرجع هم دقیقاً همین برداشت اشتباه را داشت؛ در بخش تست توضیح داده شده که چطور پیدا شد.

اتصال ماژول‌ها و خروجی‌های برد

طبق تأکید صورت آزمایش، به جای Heater و Cooler واقعی، خروجی‌ها با LED و Seven Segment نمایش داده می‌شوند. بنابراین خروجی crs علاوه بر مقدار عددی، به‌صورت کد هفت‌قطعه‌ای (crs_seg) هم بیرون داده می‌شود تا طراحی بدون منطق واسط اضافه روی برد قابل استفاده باشد.

سیگنال	جهت	پهنا	توضیح
Clk	ورودی	۱	کلاک برد
RstN	ورودی	۱	ریست ناهم‌گام فعال‌پایین
temp	ورودی	۸	دمای حسگر، علامت‌دار، ۱۰- تا ۶۰+
heater	خروجی	۱	LED گرم‌کن
cooler	خروجی	۱	LED خنک‌کن
crs	خروجی	۴	دور پیکه بر حسب RPS (۰/۴/۶/۸)
crs_seg	خروجی	۷	همان مقدار، کدشده برای Seven Segment

```

1  `include "incubator_pkg.vh"
2
3  // Incubator digital control unit - TOP LEVEL.
4  //
5  // The lab asks for the *digital control unit* only: the sensor, heater and
6  // cooler are outside the chip. The temperature therefore arrives as an
7  // 8-bit number and the actuators are represented by output signals which,
8  // on the FPGA board, drive LEDs and a seven-segment display instead of real
9  // hardware.
10 //
11 //      temp[7:0]                main_fsm    heater                cooler
12 //                                     (state==S2)
13 //
14 //
15 //      fan_fsm    crs[3:0]
16 //                                     (enable)
17 //
18 //      tick_gen    tick (once per minute)    both FSMs
19 //
20 // Board-friendly outputs
21 // -----
22 // heater / cooler : drive two LEDs.
23 // crs              : the fan speed in RPS as a plain number (0/4/6/8), ready
24 //                   for a seven-segment decoder.
25 // crs_seg          : the same value already encoded for a common-anode
26 //                   seven-segment digit, so the design can be dropped onto
27 //                   the board without extra glue.
28 module incubator_controller #(
29     // Clocks between temperature samples. Override in simulation.
30     parameter integer TICK_DIVIDE = 60_000_000
31 ) (
32     input wire          Clk,
33     input wire          RstN,          // async, active-low
34     input wire signed [7:0] temp,      // from the sensor, -10..+60 C
35
36     output wire          heater,        // LED / heater enable
37     output wire          cooler,        // LED / cooler enable
38     output wire [3:0]    crs,           // cooler speed in RPS (0/4/6/8)
39     output wire [6:0]    crs_seg,       // seven-segment (common anode)
40     output wire [1:0]    main_state,    // debug / waveform visibility
41     output wire [1:0]    fan_state,
42     output wire          tick
43 );
44 wire          cooler_next;
45 wire [1:0]    m_state;
46
47 tick_gen #(TICK_DIVIDE) u_tick (
48     .Clk(Clk), .RstN(RstN), .tick(tick)
49 );
50
51 main_fsm u_main (
52     .Clk          (Clk),
53     .RstN         (RstN),
54     .tick         (tick),
55     .temp         (temp),
56     .heater        (heater),
57     .cooler        (cooler),
58     .state        (m_state),
59     .cooler_next  (cooler_next)
60 );
61
62 // The fan machine runs only while the cooler is on. It is gated on the
63 // main FSM's NEXT state so that the tick which switches the cooler off
64 // also parks the fan in OUT -- otherwise the speed display would show a
65 // turning fan for one whole sample period after the cooler stopped.
66
67 fan_fsm u_fan (
68     .Clk          (Clk),

```

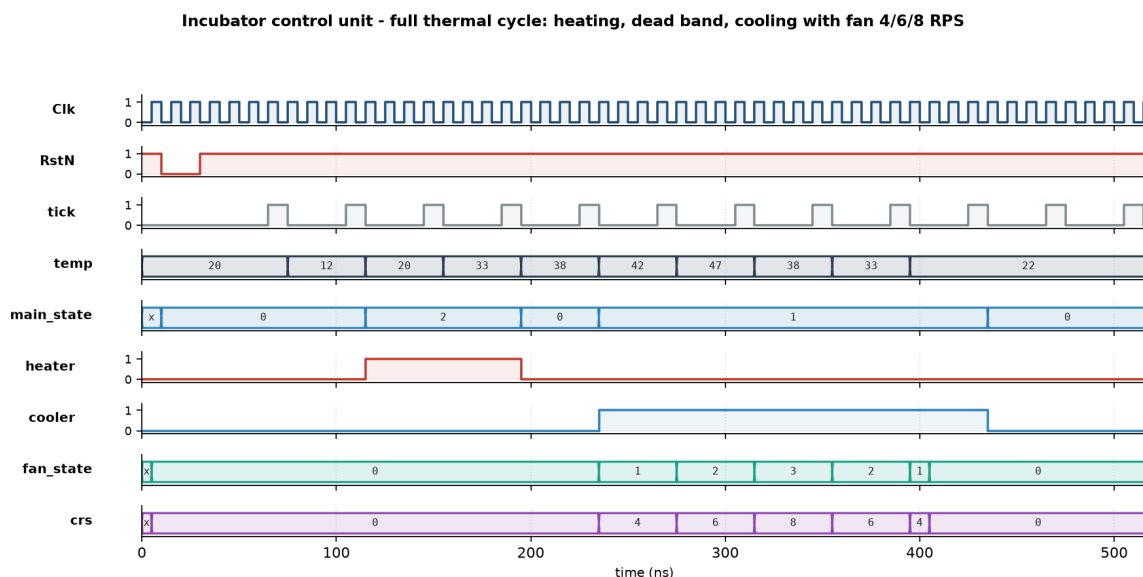
```

69 .RstN (RstN),
70 .tick (tick),
71 .temp (temp),
72 .enable (cooler_next),
73 .crs (crs),
74 .state (fan_state)
75 );
76
77 assign main_state = m_state;
78
79 seven_seg_decoder u_seg (.value(crs), .seg(crs_seg));
80 endmodule

```

شکل موج

شکل موج یک چرخه‌ی کامل حرارتی را نشان می‌دهد: از باند مرده شروع می‌شود، به سرما می‌رود و گرم‌کن روشن می‌شود، دوباره گرم می‌شود تا خنک‌کن روشن شود و پنبه نردبان ۸ → ۶ → ۴ را بالا و پایین برود، و در پایان همه‌چیز خاموش می‌شود.



شکل ۱ - چرخه‌ی کامل کنترل دما (مقادیر temp بر حسب درجه‌ی سانتیگراد)

نکات قابل مشاهده: در دمای ۱۲ درجه گرم‌کن روشن می‌شود و در ۲۰ درجه همچنان روشن می‌ماند — این همان خاصیت پسماند است، نه اشکال. در ۳۳ درجه گرم‌کن خاموش می‌شود و در ۳۸ درجه خنک‌کن روشن شده و پنبه بلافاصله روی ۴ RPS قرار می‌گیرد. سپس با ۴۲ و ۴۷ درجه به ۶ و ۸ می‌رسد و در مسیر برگشت دوباره پایین می‌آید. در پایان با رسیدن دما به ۲۲ درجه، cooler پایین می‌آید و در همان لحظه مقدار crs صفر می‌شود؛ یعنی همان رفتاری که در بخش قبل درباره‌اش صحبت شد.

تست بنچ اول: مقایسه با مدل مرجع

تست بنچ اصلی یک مدل مرجع مستقل از هر دو نمودار حالت نگه می‌دارد و بعد از هر نمونه‌برداری، حالت هر دو ماشین به همراه خروجی‌های heater، cooler و crs را با آن مقایسه می‌کند.

پوشش تست شامل این موارد است: ریست، باند مرده‌ی ۱۵ تا ۳۵ درجه، چرخه‌ی گرمایش، چرخه‌ی سرمایش با کل نردبان پنبه، ریست وسط کار، و مهم‌تر از همه یک جاروب کامل: از چهار پیکربندی مختلف (main, fan) شروع می‌شود و از هر کدام تمام دماهای بازه‌ی ۱۰- تا ۶۰+ اعمال و با مدل مقایسه می‌شود.

علاوه بر این، یک ناوردا (invariant) به صورت پیوسته بررسی می‌شود:

```
// crs must never be non-zero while the cooler is off
always @(posedge Clk)
    if (RstN && !cooler && crs != 4'd0) ... // FAIL
```

این بررسی در هر لبه‌ی کلاک انجام می‌شود، نه فقط در نقاط نمونه‌برداری؛ بنابراین حتی یک لحظه چرخش پنکه در زمان خاموشی خنک‌کن هم تست را رد می‌کند.

```
1 `timescale 1ns/1ps
2 `include "incubator_pkg.vh"
3
4 // Self-checking testbench for the incubator control unit.
5 //
6 // A reference model reimplements both state diagrams independently of the
7 // RTL, and the two are compared after every sample tick. The stimulus walks
8 // the whole legal temperature range so that every guard in both diagrams is
9 // exercised, including the "no guard is satisfied -> hold" case.
10 module tb_incubator;
11     // Small divider so a "minute" is 4 clocks in simulation.
12     localparam integer TICKDIV = 4;
13
14     reg          Clk, RstN;
15     reg signed [7:0] temp;
16
17     wire          heater, cooler, tick;
18     wire [3:0]    crs;
19     wire [6:0]    crs_seg;
20     wire [1:0]    main_state, fan_state;
21
22     incubator_controller #(TICK_DIVIDE(TICKDIV)) dut (
23         .Clk(Clk), .RstN(RstN), .temp(temp),
24         .heater(heater), .cooler(cooler), .crs(crs), .crs_seg(crs_seg),
25         .main_state(main_state), .fan_state(fan_state), .tick(tick)
26     );
27
28     initial Clk = 1'b0;
29     always #5 Clk = ~Clk;
30
31     integer pass, fail;
32     integer t, k;
33
34     // ----- reference model -----
35     localparam [1:0] R_S1 = 2'd0, R_S2 = 2'd1, R_S3 = 2'd2;
36     localparam [1:0] R_OUT = 2'd0, R_F1 = 2'd1, R_F2 = 2'd2, R_F3 = 2'd3;
37
38     reg [1:0] ref_main, ref_fan;
39
40     task automatic ref_reset;
41     begin
42         ref_main = R_S1;
43         ref_fan = R_OUT;
44     end
45     endtask
46
47     // Advance the model by one sample, exactly as the diagrams describe.
48     task automatic ref_step;
49     input signed [7:0] T;
50     reg [1:0] nm;
51     begin
52         // ---- main diagram ----
53         nm = ref_main;
54         case (ref_main)
55             R_S1: begin
56                 if (T > 35) nm = R_S2;
57                 else if (T < 15) nm = R_S3;
58             end
59             R_S2: if (T < 25) nm = R_S1;
60             R_S3: if (T > 30) nm = R_S1;
61         endcase
62
63         // ---- fan diagram ----
64         // Gated on the main FSM's NEXT state: the tick that switches the
65         // cooler off must also park the fan, and the tick that switches
66         // it on lets the fan take its first step in the same sample.
67         if (nm != R_S2) begin
68             ref_fan = R_OUT;
69         end else begin
70             case (ref_fan)
71                 R_OUT: if (T > 35) ref_fan = R_F1;
72                 R_F1: begin
73                     if (T > 40) ref_fan = R_F2;
74                     else if (T < 25) ref_fan = R_OUT;
75                 end
76                 R_F2: begin
```



```

77         if (T > 45) ref_fan = R_F3;
78         else if (T < 35) ref_fan = R_F1;
79     end
80     R_F3: if (T < 40) ref_fan = R_F2;
81 endcase
82 end
83
84     ref_main = nm;
85 end
86 endtask
87
88 function [3:0] ref_crs;
89     input [1:0] fs;
90     begin
91         case (fs)
92             R_F1: ref_crs = 4'd4;
93             R_F2: ref_crs = 4'd6;
94             R_F3: ref_crs = 4'd8;
95             default: ref_crs = 4'd0;
96         endcase
97     end
98 endfunction
99
100 // ----- helpers -----
101 task automatic check;
102     input [255:0] tag;
103     reg exp_heater, exp_cooler;
104     begin
105         exp_heater = (ref_main == R_S3);
106         exp_cooler = (ref_main == R_S2);
107         if (main_state == ref_main && fan_state == ref_fan &&
108             heater == exp_heater && cooler == exp_cooler &&
109             crs == ref_crs(ref_fan)) begin
110             pass = pass + 1;
111         end else begin
112             $display("FAIL [%0s] T=%0d | main got=%0d exp=%0d | fan got=%0d exp=%0d | H=%b/%b C=%b/%b
113             crs=%0d/%0d",
114                 tag, temp, main_state, ref_main, fan_state, ref_fan,
115                 heater, exp_heater, cooler, exp_cooler,
116                 crs, ref_crs(ref_fan));
117             fail = fail + 1;
118         end
119     end
120 endtask
121
122 // Hold `temp` and let exactly one sample tick be consumed.
123 task automatic sample;
124     input signed [7:0] T;
125     input [255:0] tag;
126     begin
127         temp = T;
128         @(posedge tick); // the edge on which the FSMs update
129         @(posedge Clk); // let the new state settle
130         #1;
131         ref_step(T);
132         check(tag);
133     end
134 endtask
135
136 task automatic do_reset;
137     begin
138         @(negedge Clk);
139         RstN = 1'b0;
140         temp = 8'sd20;
141         @(negedge Clk);
142         @(negedge Clk);
143         RstN = 1'b1;
144         ref_reset;
145         #1;
146         check("reset");
147     end
148 endtask
149
150 initial begin
151     pass = 0; fail = 0;
152     RstN = 1'b1; temp = 8'sd20;
153     ref_reset;
154
155     do_reset;
156
157     // 1) Reset really lands in S1 with everything off.
158     if (heater == 1'b0 && cooler == 1'b0 && crs == 4'd0 &&
159         main_state == R_S1 && fan_state == R_OUT)
160         pass = pass + 1;
161     else begin
162         $display("FAIL: wrong reset outputs");
163         fail = fail + 1;

```

```

163     end
164
165     // 2) Dead band: inside 15..35 nothing should ever switch on.
166     for (t = 15; t <= 35; t = t + 1)
167         sample(t[7:0], "deadband_hold");
168
169     // 3) Heating cycle: drop below 15, warm back through the band.
170     sample(8'sd14, "heat_on"); // S1 -> S3
171     sample(-8'sd10, "heat_stay_min"); // coldest reading, still heating
172     sample(8'sd20, "heat_hold"); // 15..30: no guard -> stay in S3
173     sample(8'sd30, "heat_hold_edge"); // T>30 is strict: 30 must hold
174     sample(8'sd31, "heat_off"); // S3 -> S1
175
176     // 4) Cooling cycle and the whole fan ladder.
177     sample(8'sd35, "cool_edge_hold"); // T>35 strict: 35 must NOT start
178     sample(8'sd36, "cool_on"); // S1 -> S2, fan enabled from OUT
179     sample(8'sd36, "fan_out_to_s1"); // OUT -> F_S1 (4 RPS)
180     sample(8'sd41, "fan_s1_to_s2"); // F_S1 -> F_S2 (6 RPS)
181     sample(8'sd46, "fan_s2_to_s3"); // F_S2 -> F_S3 (8 RPS)
182     sample(8'sd60, "fan_max_temp"); // hottest reading, stay at 8 RPS
183     sample(8'sd39, "fan_s3_to_s2"); // T<40 -> back to 6 RPS
184     sample(8'sd34, "fan_s2_to_s1"); // T<35 -> back to 4 RPS
185     sample(8'sd30, "fan_s1_hold"); // 25..35: fan holds, cooler holds
186     sample(8'sd24, "cool_off"); // T<25: main -> S1, fan -> OUT
187
188     // 5) Leaving S2 must park the fan in OUT, and re-entry restarts there.
189     sample(8'sd36, "reenter_cool"); // back to S2
190     sample(8'sd46, "reenter_fan1"); // OUT -> F_S1 even though T>45
191     sample(8'sd46, "reenter_fan2"); // F_S1 -> F_S2
192     sample(8'sd46, "reenter_fan3"); // F_S2 -> F_S3
193     sample(8'sd10, "cool_off_again"); // straight out of S2
194
195     // 6) Reset in the middle of cooling.
196     sample(8'sd50, "pre_reset_cool");
197     sample(8'sd50, "pre_reset_fan");
198     do_reset;
199
200     // 7) Exhaustive: from every reachable (main, fan) pair, apply every
201     // temperature in range and compare against the model.
202     for (k = 0; k < 4; k = k + 1) begin
203         // Drive the machine into a known (main, fan) configuration.
204         do_reset;
205         case (k)
206             0: ; // stay in S1
207             1: begin sample(8'sd14, "setup"); end // S3, heater on
208             2: begin sample(8'sd36, "setup"); end // S2 + fan F_S1
209                 sample(8'sd36, "setup"); end
210             3: begin sample(8'sd36, "setup"); end // S2 + fan F_S3
211                 sample(8'sd36, "setup");
212                 sample(8'sd41, "setup");
213                 sample(8'sd46, "setup"); end
214         endcase
215         for (t = -10; t <= 60; t = t + 1)
216             sample(t[7:0], "sweep");
217     end
218
219     // 8) Seven-segment output must track `crs`.
220     do_reset;
221     sample(8'sd36, "seg_setup");
222     sample(8'sd36, "seg_4rps");
223     if (crs == 4'd4 && crs_seg == 7'b0011001) pass = pass + 1;
224     else begin
225         $display("FAIL: seg for 4 RPS = %b", crs_seg);
226         fail = fail + 1;
227     end
228
229     $display("tb_incubator: Passed: %0d, Failed: %0d", pass, fail);
230     if (fail != 0) $fatal(1);
231     $finish;
232 end
233
234 // ---- standing invariant -----
235 // The lab text is explicit that a rotation speed is meaningless while
236 // the cooler is off. This is checked continuously, not just at chosen
237 // sample points, so any glitch of `crs` while `cooler` is low fails the
238 // run. (This is exactly the class of bug that a next-state gating error
239 // produces, and it is why the check is written as an invariant.)
240 always @(posedge Clk) begin
241     if (RstN && !cooler && crs != 4'd0) begin
242         $display("FAIL: crs=%0d while cooler is OFF (T=%0d, fan_state=%0d)",
243             crs, temp, fan_state);
244         fail = fail + 1;
245     end
246 end
247
248 initial begin
249     #2000000;

```

```

250     $display("FAIL: timeout");
251     $fatal(1);
252 end
253 endmodule

```

تست پنجم دوم: انطباق با نمودار

مدل مرجع بررسی قدرتمندی است، اما یک ضعف ذاتی دارد: مدلی که به دست همان طراح نوشته شده، ممکن است همان برداشت اشتباه از صورت مسئله را تکرار کند. دقیقاً همین اتفاق در این آزمایش افتاد — هم RTL و هم مدل مرجع، در ابتدا اجازه می‌دادند پیکه در لحظه‌ی خاموش شدن خنک‌کن یک قدم اضافه بردارد و هر دو با هم موافق بودند، پس تست سبز بود.

به همین دلیل یک تست پنجم دوم نوشته شد که مستقیماً از روی نمودارهای صورت آزمایش و بدون هیچ مدل مشترکی، فهرستی از رفتارهایی را که باید برقرار باشند بررسی می‌کند: مقایسه‌های اکید روی هر شش آستانه، پله‌های ۴/۶/۸، صفر بودن سرعت در تمام حالت‌هایی که خنک‌کن خاموش است، و شروع دوباره از پایین نردبان پس از بازگشت به S_۲. همین تست بود که اشکال را پیدا کرد.

```

1  `timescale 1ns/1ps
2  // Diagram-conformance testbench.
3  //
4  // tb_incubator.v checks the design against a reference model. That is a
5  // strong check, but a model written by the same hand can repeat the same
6  // misreading of the spec -- which is exactly what happened here: both RTL
7  // and model initially let the fan take one extra step on the tick that
8  // switched the cooler off.
9  //
10 // This testbench is therefore written directly from the two state diagrams
11 // in the lab sheet, as a list of behaviours that must hold, with no shared
12 // model. It is the check that caught that bug.
13 module tb_incubator_spec;
14     localparam integer TD = 4;
15     reg Clk=0, RstN=1; reg signed [7:0] temp=8'sd20;
16     wire heater,cooler,tick; wire [3:0] crs; wire [6:0] seg;
17     wire [1:0] ms, fs;
18     incubator_controller #(TICK_DIVIDE(TD)) dut(.Clk(Clk),.RstN(RstN),.temp(temp),
19         .heater(heater),.cooler(cooler),.crs(crs),.crs_seg(seg),
20         .main_state(ms),.fan_state(fs),.tick(tick));
21     always #5 Clk=~Clk;
22     integer bad=0, checks=0;
23     task automatic step(input signed [7:0] T);
24     begin temp=T; @(posedge tick); @(posedge Clk); #1; end endtask
25     task automatic want(input [255:0] w, input ok);
26     begin checks=checks+1; if(!ok) begin $display(" MISMATCH: %0s (T=%0d ms=%0d fs=%0d H=%b C=%b crs=%0d)",w
27         ,temp,ms,fs,heater,cooler,crs); bad=bad+1; end end endtask
28     task automatic rst; begin @(negedge Clk); RstN=0; @(negedge Clk); @(negedge Clk); RstN=1; #1; end endtask
29     initial begin
30         rst;
31         $display("A) reset => S1, all off");
32         want("reset S1 off", ms==0 && !heater && !cooler && crs==0);
33
34         $display("B) strict inequalities on the boundaries");
35         rst; step(8'sd35); want("T=35 must NOT turn cooler on", !cooler);
36         rst; step(8'sd36); want("T=36 turns cooler on", cooler);
37         rst; step(8'sd15); want("T=15 must NOT turn heater on", !heater);
38         rst; step(8'sd14); want("T=14 turns heater on", heater);
39         rst; step(8'sd14); step(8'sd30); want("T=30 keeps heater on", heater);
40         rst; step(8'sd14); step(8'sd31); want("T=31 turns heater off", !heater);
41         rst; step(8'sd36); step(8'sd25); want("T=25 keeps cooler on", cooler);
42         rst; step(8'sd36); step(8'sd24); want("T=24 turns cooler off", !cooler);
43
44         $display("C) fan ladder speeds are 4/6/8 RPS");
45         rst; step(8'sd36); step(8'sd36); want("fan 4RPS", crs==4);
46         step(8'sd41); want("fan 6RPS", crs==6);
47         step(8'sd46); want("fan 8RPS", crs==8);
48         step(8'sd39); want("back to 6RPS", crs==6);
49         step(8'sd34); want("back to 4RPS", crs==4);
50
51         $display("D) fan is meaningless while cooler is off => crs must be 0");
52         rst; want("crs=0 in S1", crs==0);
53         rst; step(8'sd14); want("crs=0 while heating (S3)", crs==0 && heater);
54         // even at a very hot reading, if main is not in S2 the fan must be OUT
55         rst; step(8'sd60); step(8'sd60); step(8'sd60);
56         want("hot => cooler on, fan running", cooler && crs>0);
57         step(8'sd10); // drops out of S2
58         want("leaving S2 parks fan at 0", !cooler && crs==0 && fs==0);

```

```

58
59 $display("E) re-entering S2 restarts the fan from OUT (not from 8RPS)");
60 // On the entry tick the main FSM moves S1->S2 and the fan's own
61 // OUT->S1 guard (T>35) is satisfied by the same reading, so the cooler
62 // starts at its lowest speed on that tick. The point being checked is
63 // that it restarts from the BOTTOM of the ladder, not from the 8 RPS it
64 // held before it was parked.
65 step(8'sd60);
66 want("re-entry: fan restarts at 4RPS (not 8)", cooler && crs==4);
67 step(8'sd60);
68 want("re-entry: then climbs to 6RPS", crs==6);
69
70 $display("F) hold behaviour inside the dead band");
71 rst; step(8'sd20); want("T=20 from reset stays S1", ms==0 && !heater && !cooler);
72 rst; step(8'sd14); step(8'sd20); want("T=20 while heating keeps heating", heater);
73 rst; step(8'sd36); step(8'sd30); want("T=30 while cooling keeps cooling", cooler);
74
75 $display("tb_incubator_spec: Passed: %0d, Failed: %0d", checks - bad, bad);
76 if (bad != 0) $fatal(1);
77 $finish;
78 end
79 endmodule

```

نتایج تست

همه‌ی شبیه‌سازی‌ها با Icarus Verilog 12.0:

نتیجه	پوشش	تست
۳۴۵/۳۴۵	مدل مرجع + جاروب کامل دما از چهار پیکربندی + ناوردای پیوسته	tb_incubator
۲۳/۲۳	انطباق مستقیم با نمودارهای صورت آزمایش	tb_incubator_spec

خروجی واقعی run_tests.sh:

```

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_incubator ===
tb_incubator: Passed: 345, Failed: 0
PASS: tb_incubator

=== tb_incubator_spec ===
tb_incubator_spec: Passed: 23, Failed: 0
PASS: tb_incubator_spec

summary: 2/2 passed, 0 failed

```

جمع‌بندی

در این آزمایش دو ماشین حالت طراحی شد که یکی از آن‌ها مشروط به وضعیت دیگری فعال می‌شود. تجربه‌ی اصلی این بود که وقتی دو ماشین حالت روی یک رویداد مشترک (اینجا پالس نمونه‌برداری) کار می‌کنند، باید دقیقاً مشخص کرد که هر کدام بر اساس کدام نسخه از حالت دیگری تصمیم می‌گیرد؛ حالت فعلی یا حالت بعدی. انتخاب اشتباه، مداری می‌سازد که نه در شبیه‌سازی خطای نحوی می‌دهد و نه لزوماً در تست‌های ساده دیده می‌شود، ولی رفتار فیزیکی نادرستی تولید می‌کند.

درس دوم درباره‌ی روش تست بود. مقایسه با مدل مرجع تنها تا جایی معتبر است که مدل، برداشت درستی از صورت مسئله داشته باشد. نوشتن یک تست دوم مستقیماً از روی مشخصات و بدون مدل مشترک، همان چیزی بود که اشکال را آشکار کرد.

در نهایت، خاصیت پسماند در این طراحی یک ویژگی عمدی است نه اتفاقی. باند مرده‌ی ۱۵ تا ۳۵ درجه باعث می‌شود سیستم حول یک نقطه‌ی تنظیم دچار روشن و خاموش شدن پیاپی نشود، که هم برای عمر عملگرها و هم برای پایداری دمای داخل محفظه مطلوب است.